SQL Cab Booking System

Professional Project Report | SQL Database Management & Analysis

Prepared by: Chetana Wagh

Project type: SQL portfolio / learning project

1. Executive Summary

The SQL Cab Booking System models essential cab booking operations and demonstrates analytical SQL queries. The supplied screenshots show customer, driver, cab, booking and feedback tables, plus queries for completed trips, cancellation percentages and booking trends.

This report documents the demonstrated design and analytical intent. It does not claim unprovided query output or business performance results.

2. Project Objectives

Manage customer, driver, cab and trip records; enforce relationships through primary and foreign keys; analyze completed trips, cancellation rates and day-of-week demand; and store trip feedback.

3. Database Architecture

The Bookings table acts as the central transaction table. Each booking refers to a customer, driver and cab. Feedback references a booking. These foreign-key relationships help preserve referential integrity.

Table	Purpose / key relationship
Customers	Customer identifiers and names
Drivers	Driver identifiers, names and ratings
Cabs	Cab identifiers and types
Bookings	Trips; references Customers, Drivers, Cabs
Feedback	Ratings/comments; references Bookings

4. SQL Techniques

The shared code demonstrates CREATE DATABASE, CREATE TABLE, INSERT INTO, PRIMARY KEY, FOREIGN KEY, SELECT, WHERE, COUNT, SUM, GROUP BY, ORDER BY, HAVING, CASE WHEN, DAYNAME and DAYOFWEEK.

5. Analysis & Business Questions

Completed bookings: count completed trips for each customer and sort to identify frequent riders.

Cancellation analysis: compute cancelled bookings divided by all bookings per customer, then filter rates over 30%.

Weekday demand: group booking timestamps by weekday name and count bookings.

```
SELECT customer_id, COUNT(*) AS total_bookings
FROM Bookings WHERE status = 'Completed'
GROUP BY customer_id ORDER BY total_bookings DESC;

SELECT customer_id, COUNT(*) AS total_bookings,
SUM(CASE WHEN status='Cancelled' THEN 1 ELSE 0 END) AS cancelled_bookings,
100.0 * SUM(CASE WHEN status='Cancelled' THEN 1 ELSE 0 END)/COUNT(*) AS cancel_percentage
FROM Bookings GROUP BY customer_id HAVING cancel_percentage > 30;

SELECT DAYNAME(booking_time) AS day, COUNT(*) AS total_bookings
FROM Bookings GROUP BY day ORDER BY total_bookings DESC;
```

6. Data Quality & Validation

Execute the complete SQL file in the intended MySQL database. Verify table creation and inserted rows, confirm foreign-key references, and check the database selected by USE matches the created database.

The GitHub screenshot shows an uploaded file named SQL PROJECT.sql. The earlier screenshots illustrate a similar cab-booking schema, but may not be an exact line-by-line representation of the uploaded file.

7. Portfolio & Future Enhancements

GitHub repository: SQL-Cab-Booking-System. Key files: README.md and SQL PROJECT.sql. Add screenshots of actual query results if available.

Potential improvements include fare/revenue breakdown by cab type, average trip distance, monthly demand trends, driver rating summaries and a dashboard built from validated query outputs.

8. Conclusion

The project provides evidence of foundational relational database design and practical SQL analytics. Its strongest portfolio value comes from the documented schema, executable code and clearly explained business questions.